

Máster en Ciberseguridad
2025-2026

Trabajo Fin de Máster

“Desarrollo de un Bot Basado en LLMs para la Detección y Alerta Temprana de Mensajes Potencialmente de Phishing”

Diego Montero Solano

Tutor/es

Eduardo Cibrián Sánchez

Madrid, 2026



Esta obra se encuentra sujeta a la licencia Creative Commons
Reconocimiento – No Comercial – Sin Obra Derivada

RESUMEN

El presente trabajo de Fin de Master tiene como objetivo el diseño, desarrollo e implementación de un sistema automatizado de detección y alerta temprana de mensajes potencialmente fraudulentos, basado en modelos de lenguaje de gran tamaño (LLMs). El sistema integra distintos proveedores de inteligencia artificial y permite su despliegue multiplataforma mediante bots capaces de analizar mensajes en tiempo real.

La arquitectura desarrollada se basa en un diseño modular que separa la gestión de configuración, la interacción con plataformas de mensajería y la capa de análisis inteligente. Se ha implementado un módulo de abstracción que permite utilizar diferentes proveedores de LLMs como OpenAI, Gemini o Claude, garantizando flexibilidad y escalabilidad. Asimismo, se ha aplicado un proceso de optimización del prompt siguiendo buenas prácticas de Prompt Engineering, con el objetivo de transformar un modelo generativo en un clasificador estructurado de riesgo de phishing.

El sistema clasifica los mensajes en distintos niveles de riesgo (bajo, medio, y alto) y genera alertas contextualizadas para el usuario final, favoreciendo la concienciación y la prevención de ataques. La solución propuesta demuestra la viabilidad de emplear modelos de lenguaje avanzados como herramienta complementaria en sistemas de defensa frente a amenazas basadas en ingeniería social.

Palabras clave: phishing, bot, Inteligencia Artificial, IA, amenazas, ciberseguridad, aplicación de mensajería, Telegram, Discord, Python, Large Language Model (LLM), prompt, prompt engineering, detección de amenazas, ingeniería social.

Índice de Contenido

1. Introducción.....	9
1.1. Motivación del Trabajo.....	10
1.2. Objetivos.....	11
1.3. Marco Regulador	12
1.4. Entorno Socioeconómico	14
2. Estado de la Cuestión	15
2.1. Definiciones	16
2.2. Situación Actual.....	17
2.3. Trabajos Relacionados	18
2.4. Diseño de Soluciones.....	20
2.5. Herramientas y Tecnologías Utilizadas	20
3. Análisis del Sistema	21
3.1. Descripción	21
3.2. Requisitos.....	22
3.3. Diagrama de Flujo	24
4. Diseño e Implementación de la Solución	25
4.1. Creación de un Bot para Telegram	25
4.2. Creación de un Bot para Discord	26
4.3. Desarrollo de una Herramienta para la Gestión de Bots.....	27
4.4. Inclusión de Técnicas de Prompt Engineering	30
5. Pruebas y Resultados	32
5.1. Pruebas sobre el LLM.....	33
5.2. Otras Pruebas	36
6. Conclusiones.....	37
6.1. Objetivos Cumplidos	38
6.2. Líneas de Trabajo Futuras.....	39

Índice de Figuras

Figura 1: Resumen vectores de ataque en accesos - Informe Verizon [1]	10
Figura 2: Diagrama de Flujo del Sistema	24
Figura 3: Creación de bot de Telegram con FatherBot	25
Figura 4: Creación de Bot de Discord	27
Figura 5: Prompt final después del prompt engineering.....	32
Figura 6: Ejemplo mensaje riesgo alto Telegram.....	33
Figura 7: Ejemplo mensaje riesgo alto Discord.....	34
Figura 8: Prompt injection fallido	35
Figura 9: Prompt injection exitoso	36

1. Introducción

En los últimos años, el crecimiento exponencial de las plataformas de mensajería instantánea y comunicación en tiempo real ha transformado la manera en que las personas y organizaciones interactúan. Aplicaciones como Whatsapp, Telegram, Discord, Microsoft Teams o Messenger se han consolidado como canales habituales tanto para la comunicación personal como para la coordinación profesional, comunidades técnicas y entornos educativos. Sin embargo, este incremento en la conectividad ha venido acompañado de un aumento significativo de amenazas de ciberseguridad, especialmente aquellas relacionadas con la ingeniería social.

Entre estas amenazas, el phishing continúa siendo una de las técnicas más utilizadas por los atacantes. A través de mensajes que simulan provenir de entidades legítimas, los ciberdelincuentes buscan obtener credenciales, información financiera o datos sensibles de los usuarios. Tradicionalmente, las soluciones de detección de phishing se han centrado en el análisis de correos electrónicos o en la inspección de URLs, pero el auge de las plataformas de mensajería ha ampliado la superficie de ataque, requiriendo nuevas aproximaciones adaptadas a estos entornos.

Paralelamente, el avance de la Inteligencia artificial y los Grandes Modelos Lingüísticos (LLM) de IA Generativa ha abierto nuevas posibilidades en el análisis semántico y contextual de textos. Estos modelos permiten comprender no solo patrones superficiales, sino también intenciones, matices lingüísticos y estructuras discursivas complejas, lo que los convierte en una herramienta prometedora para la detección de mensajes potencialmente maliciosos.

En este contexto, el presente Trabajo de Fin de Master propone el desarrollo de un sistema basado en LLMs capaz de integrarse como un bot en plataformas de mensajería, con el objetivo de detectar y alertar de forma temprana sobre mensajes potencialmente de phishing, contribuyendo así a la mejora de la seguridad en comunidades digitales.

1.1. Motivación del Trabajo

El phishing sigue siendo una de las principales causas de incidentes de seguridad, tanto en entornos corporativos como en el ámbito particular. A pesar de las campañas de concienciación y de la mejora de los filtros tradicionales, los atacantes continúan perfeccionando sus técnicas, empleando mensajes cada vez más personalizados y convincentes. Diversos informes recientes evidencian que el phishing continúa siendo uno de los vectores de ataque más relevantes en la actualidad. En este sentido, el informe *Data Breach Investigations Report* de Verizon correspondiente a 2024 señala que el phishing está presente en un porcentaje significativo de brechas de seguridad analizadas, consolidándose como una de las técnicas más empleadas para el acceso inicial a sistemas corporativos [1]. De manera complementaria, el informe anual de INCIBE destaca un incremento sostenido en los incidentes relacionados con ingeniería social en España, especialmente aquellos dirigidos a usuarios finales a través del fraude online, destacando 21571 casos de phishing en esta categoría [2]. Estos datos ponen de manifiesto no solo la persistencia del problema, sino también su evolución hacia nuevos entornos digitales.

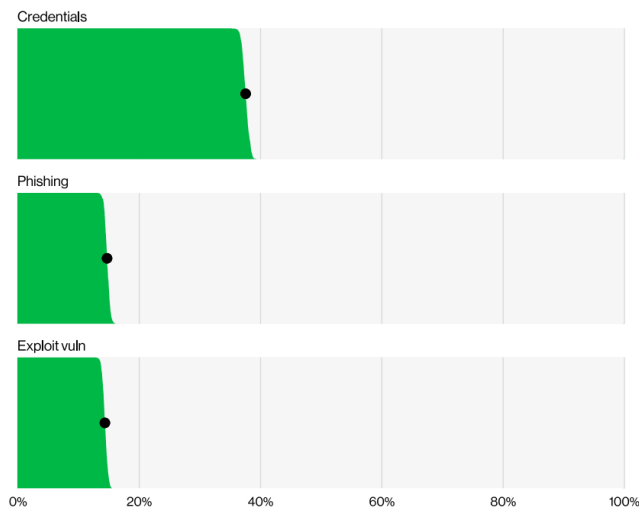


Figura 1: Resumen vectores de ataque en accesos - Informe Verizon [1]

La motivación principal de este trabajo surge de la necesidad de trasladar los mecanismos de protección más allá del correo electrónico, extendiéndolos a plataformas de mensajería instantánea como Telegram o Discord, que se han convertido en entornos habituales para comunidades técnicas, grupos de inversión, proyectos de desarrollo software, servidores educativos y equipos de trabajo distribuidos. A diferencia de los filtros clásicos basados en reglas estáticas o listas negras de URLs, los LLMs ofrecen una capacidad de análisis

contextual que permite evaluar la intención comunicativa del mensaje, la urgencia artificial, la solicitud de datos sensibles o la presencia de patrones típicos de ingeniería social. Esto abre la puerta a sistemas de detección más flexibles, adaptativos y capaces de enfrentarse a nuevas formas de ataque.

Asimismo, resulta relevante destacar que, a diferencia del correo electrónico y otros mecanismos de comunicación, donde existen múltiples capas de seguridad consolidadas, como filtros antispam, sistemas de reputación de dominios o pasarelas de seguridad corporativas, las aplicaciones de mensajería instantánea carecen en muchos casos de mecanismos equivalentes de filtrado automático. Plataformas como Telegram o Discord no disponen por defecto de una carpeta de spam ni de sistemas avanzados de detección previa de amenazas, lo que incrementa la exposición del usuario a mensajes potencialmente maliciosos. Esta ausencia de controles preventivos convierte a estos entornos en un vector especialmente atractivo para los atacantes y refuerza la necesidad de desarrollar soluciones específicas adaptadas a sus características.

1.2. Objetivos

El objetivo principal de este Trabajo de Fin de Máster es el diseño y desarrollo de un framework modular para la detección precoz de amenazas de phishing en plataformas de mensajería instantánea, basado en el uso de Modelos de Lenguaje de Gran Escala (LLMs). Este framework pretende proporcionar una solución flexible, extensible y adaptable que permita analizar mensajes en tiempo real y generar alertas tempranas ante posibles intentos de ingeniería social, tomando como caso de estudio las plataformas de Telegram y Discord, y sentando las bases para su aplicación en otros entornos de comunicación digital.

Para alcanzar el objetivo general, se han definido los siguientes objetivos específicos:

- 1) Analizar el estado actual del phishing en plataformas de mensajería y las técnicas de detección existentes.
- 2) Diseñar una arquitectura modular basada en programación orientada a objetos que permita integrar diferentes modelos de LLM, adaptarse a distintas plataformas de mensajería y facilitar la extensibilidad y escalabilidad del sistema.

- 3) Implementar dos bots funcionales, uno para Telegram y otro para Discord.
- 4) Integrar un sistema de análisis basado en LLM que evalúe el contenido de los mensajes en tiempo real y determine su posible carácter fraudulento.
- 5) Diseñar un mecanismo de alerta comprensible para los usuarios finales, que informe del nivel de riesgo asociado a cada mensaje analizado.
- 6) Desarrollar una herramienta de despliegue sencilla que permita a cualquier usuario instalar y ejecutar los bots en su propio entorno, así como modificar el modelo de lenguaje empleado.
- 7) Evaluar el comportamiento del sistema ante distintos tipos de mensajes (legítimos, maliciosos), analizando su efectividad y limitaciones.

1.3. Marco Regulator

El desarrollo de un sistema basado en LLMs para la detección temprana de mensajes potencialmente de phishing en plataformas de mensajería debe enmarcarse dentro del contexto normativo y regulador vigente en materia de ciberseguridad, protección de datos y uso de sistemas de inteligencia artificial.

Este apartado recoge las principales normativas, directivas y guías técnicas que han sido consideradas como referencia durante el diseño e implantación del sistema.

Reglamento General de Protección de Datos (RGPD)

El Reglamento General de Protección de Datos (Reglamento (UE) 2016/679) [3] establece el marco jurídico para la protección de datos personales en la Unión Europea. Dado que el bot desarrollado analiza mensajes generados por usuarios en plataformas de mensajería, existe la posibilidad de que dichos mensajes contengan datos personales. Por ello, durante el diseño del sistema se han tenido en cuenta los siguientes principios del RGPD:

- Principio de minimización de datos: El sistema analiza únicamente el contenido necesario del mensaje para evaluar el riesgo de phishing, sin almacenar ningún tipo de información.

- Limitación de finalidad: El tratamiento de datos se realiza exclusivamente con el objetivo de detectar posibles amenazas de phishing.
- Privacidad desde el diseño y por defecto: La arquitectura modular permite desplegar el sistema en entornos locales, evitando la centralización de datos.

Directiva NIS2

La Directiva NIS2 (Directiva (UE) 2022/2555) [4] establece medidas destinadas a garantizar un elevado nivel de ciberseguridad en la Unión Europea. Si bien el sistema desarrollado no constituye en sí mismo un operador de servicios esenciales, sí puede considerarse una herramienta de apoyo a las medidas de:

- Gestión de riesgos en ciberseguridad.
- Detección temprana de incidentes.
- Prevención de ataques basados en ingeniería social.

El desarrollo del bot se alinea con el enfoque proactivo promovido por NIS2, orientado a reforzar las capacidades de detección y respuesta ante amenazas emergentes, especialmente aquellas que explotan el factor humano.

Reglamento Europeo de Inteligencia Artificial

El Reglamento Europeo de Inteligencia Artificial (Reglamento (UE) 2024/1689) [5] establece un marco regulador para los sistemas de inteligencia artificial en la Unión Europea, clasificándolos según su nivel de riesgo. El sistema desarrollado puede considerarse un sistema de IA de apoyo a la toma de decisiones en materia de ciberseguridad. Aunque no encaja en categorías de alto riesgo, sí que debe cumplir ciertos principios generales:

- Transparencia: Los usuarios serán informados de que los mensajes son analizados por un sistema automatizado.
- Supervisión humana: El bot genera alertas, pero la decisión final sobre cómo actuar recae en los usuarios o administradores del canal.
- Robustez y fiabilidad: El sistema debe minimizar falsos positivos y falsos negativos.

La arquitectura modular permite sustituir o actualizar el modelo LLM empleado, facilitando la adaptación a futuros requisitos regulatorios.

Políticas y términos de uso de plataformas

El despliegue del bot en plataformas como Telegram y Discord debe cumplir las políticas y términos de uso de dichas plataformas. En particular:

- El uso adecuado de APIs oficiales.
- Respeto a las limitaciones de rate limiting.
- Transparencia sobre el tratamiento automatizado de mensajes.
- Cumplimiento de las normas comunitarias.

El desarrollo se ha realizado utilizando las APIs oficiales de ambas plataformas, garantizando la compatibilidad con sus requisitos técnicos y normativos.

1.4. Entorno Socioeconómico

El desarrollo de soluciones orientadas a la detección de amenazas de ciberseguridad se enmarca en un contexto socioeconómico caracterizado por un incremento sostenido de la digitalización en todos los sectores productivos. Este proceso ha ido acompañado de un aumento significativo de los riesgos asociados, lo que ha impulsado a las organizaciones a incrementar su inversión en ciberseguridad como elemento estratégico para garantizar la continuidad del negocio y la protección de la información.

Según informes recientes, el gasto global en ciberseguridad ha experimentado un crecimiento constante en los últimos años. En particular, el informe de Gartner estima que la inversión mundial en ciberseguridad continúa aumentando debido al incremento de las amenazas y a la necesidad de cumplimiento normativo [6]. Además, el informe de Cybersecurity Ventures proyecta que el coste global del cibercrimen seguirá creciendo en los próximos años, lo que refuerza la necesidad de invertir en mecanismos de prevención y detección temprana [7].

En este contexto, la detección temprana de ataques de phishing resulta especialmente relevante desde un punto de vista económico. Este tipo de ataques no solo puede derivar en pérdidas financieras directas, sino también en daños reputacionales y sanciones regulatorias. De acuerdo con el informe *Data Breach Investigations Report* de Verizon

[1] mencionado anteriormente, el phishing continúa siendo uno de los principales vectores de entrada en incidentes de seguridad, lo que pone de manifiesto el impacto económico potencial de este tipo de amenazas. Por ello, el desarrollo de herramientas capaces de mitigar estos riesgos presenta un alto potencial de rentabilidad, tanto en términos de ahorro de costes derivados de los incidentes, como en la mejora de la confianza de usuarios y clientes.

Por otro lado, este trabajo se alinea con diversas iniciativas internacionales orientadas a promover un entorno digital seguro. En particular, la Organización de las Naciones Unidas, a través de la Agenda 2030, establece en sus Objetivos de Desarrollo Sostenible (ODS) la necesidad de fortalecer la resiliencia de las infraestructuras y fomentar la innovación tecnológica, así como garantizar el acceso a tecnologías seguras y fiables [8]. También cabe destacar las estrategias específicas en materia de ciberseguridad desarrolladas por la Unión Europea, como la Estrategia de Ciberseguridad de la UE, orientadas a reforzar la protección de ciudadanos y empresas frente a amenazas digitales y a fomentar el desarrollo de soluciones innovadoras en este ámbito [9].

En consecuencia, el presente trabajo no solo responde a una necesidad técnica, sino también a una demanda creciente dentro del contexto socioeconómico actual, donde la ciberseguridad se ha consolidado como un pilar fundamental para el desarrollo sostenible y la transformación digital.

2. Estado de la Cuestión

En un contexto digital donde el volumen de información y las interacciones en línea aumentan exponencialmente, la protección frente a amenazas cibernéticas se ha convertido en un desafío constante. En particular, las técnicas de ingeniería social, siendo una de las principales causas de compromisos de seguridad tanto para usuarios finales como para organizaciones. Esta sección revisa el estado actual de los bots y LLMs, su papel en los entornos de comunicación actuales, así como la evolución de los ataques de phishing y su relación con tecnologías emergentes.

2.1. Definiciones

¿Qué es un bot?

Un bot es una aplicación software automatizada diseñada para ejecutar tareas repetitivas en una red siguiendo instrucciones previamente definidas. Estos sistemas pueden imitar determinados comportamientos humanos, aunque lo hacen con mayor velocidad y precisión, y pueden funcionar de manera autónoma sin intervención humana directa [10]. En el ámbito de la mensajería instantánea, los bots pueden integrarse directamente en plataformas como Telegram o Discord para automatizar acciones como responder a comandos, moderar contenido o interactuar con usuarios en tiempo real. Estos bots aprovechan las APIs oficiales de cada plataforma para monitorizar mensajes, ejecutar lógica de negocio o generar respuestas y acciones automatizadas.

¿Qué es un LLM?

Los modelos de lenguaje de gran tamaño (LLMs) son modelos de aprendizaje profundo entrenados con volúmenes masivos de datos textuales para comprender y generar lenguaje natural. La arquitectura de transformadores en la que están basados estos modelos les permite analizar relaciones entre palabras y frases dentro de una secuencia de texto [11], permitiéndoles comprender y generar lenguaje natural con un alto grado de coherencia contextual. Gracias a esto son capaces de realizar tareas como clasificación de texto, análisis semántico, generación de respuestas y resumen automático. En los últimos años, los LLMs se han convertido en herramientas fundamentales en áreas como asistentes conversacionales, análisis de sentimiento y sistemas inteligentes de soporte, incluyendo aplicaciones de ciberseguridad.

Prompt Engineering

El prompt engineering se define como la disciplina orientada al diseño, estructuración y optimización de las instrucciones o entradas (prompts) que se proporcionan a los modelos de lenguaje con el fin de guiar su comportamiento y mejorar la calidad de sus respuestas. Según IBM, esta práctica resulta fundamental para adaptar el funcionamiento de los LLMs a tareas específicas, ya que estos modelos no son programados de manera tradicional, sino condicionados a través del lenguaje natural [12]. En este sentido, la

formulación del prompt influye directamente en la precisión, coherencia y relevancia de la salida generada.

En el ámbito de la ciberseguridad, y particularmente en la detección de phishing, el prompt engineering adquiere una importancia significativa, ya que permite orientar al modelo hacia el análisis de características concretas de los mensajes sospechosos. Entre estas características se incluyen la identificación de lenguaje persuasivo o urgente, solicitudes de información sensible, inconsistencias contextuales o patrones lingüísticos asociados a intentos de fraude. Asimismo, el uso de técnicas como *few-shot prompting*, en las que se proporcionan ejemplos previos dentro del propio prompt, o el empleo de instrucciones explícitas y estructuradas, contribuye a mejorar la capacidad del modelo para clasificar correctamente los mensajes.

De este modo, el prompt engineering no debe entenderse únicamente como un mecanismo de interacción con el modelo, sino como un componente esencial dentro del diseño del sistema, ya que condiciona directamente su rendimiento y fiabilidad. Una adecuada definición de los prompts permite maximizar el potencial de los LLMs y adaptarlos de forma eficiente a escenarios específicos como la detección temprana de amenazas en entornos de mensajería.

2.2. Situación Actual

Las plataformas de mensajería instantánea han experimentado un crecimiento significativo en los últimos años, consolidándose como uno de los principales canales de comunicación tanto en entornos personales como profesionales. Este aumento en su uso ha sido acompañado por un incremento en su explotación como vectores de ataque por parte de actores maliciosos, quienes aprovechan estas plataformas para distribuir enlaces fraudulentos, ejecutar campañas de phishing o desplegar bots automatizados con fines ilícitos.

En relación con los ataques de phishing, la tendencia mundial muestra un crecimiento sostenido. Según informes recientes, más de 709 millones de intentos de phishing y webs fraudulentas fueron bloqueados en 2023 por los sistemas de Kaspersky, representando un incremento del 40% respecto al año anterior. En 2025, los sistemas antiphishing continuaron bloqueando cientos de millones de intentos de redirección a sitios maliciosos y otras estafas [13]. Además, según otro análisis de Kaspersky, el 88% de los ataques de

phishing busca robar credenciales de acceso a cuentas, lo que demuestra la persistencia de estas amenazas como método preferido para comprometer cuentas de los usuarios [14].

Los mensajes de phishing no están restringidos al correo electrónico, y cada vez más se distribuyen a través de plataformas de mensajería. En particular, los bots han sido utilizados para facilitar la exfiltración de credenciales y propagar enlaces maliciosos. Por ejemplo, el uso de bots de Telegram como destino para exfiltrar información contenida en ataques de phishing aumentó un 800% entre 2021 y 2022, evidenciando cómo los actores maliciosos aprovechan las funcionalidades automatizadas para escalar sus técnicas [15]. Este fenómeno evidencia la capacidad de adaptación de los atacantes a nuevos entornos tecnológicos y subraya la necesidad de desarrollar soluciones específicas para estos canales de comunicación.

2.3. Trabajos Relacionados

El desarrollo de sistemas de detección de phishing basados en inteligencia artificial ha experimentado un notable crecimiento en los últimos años, impulsado en gran medida por la aparición de los modelos de lenguaje de gran tamaño (LLMs). A continuación, se revisan algunos de los trabajos más relevantes relacionados con el uso de LLMs para la detección de phishing.

Detección de phishing mediante LLMs

Uno de los estudios de referencia en este ámbito es el de Heiding et al., titulado “Devising and Detecting Phishing Emails Using Large Language Models” y publicado en IEEE Access en 2024 [16]. El trabajo analiza de forma simultánea la capacidad de los LLMs tanto para generar como para detectar correos de phishing. Sus experimentos demuestran que modelos como GPT-4 superan a los filtros tradicionales basados en reglas en tareas de clasificación de mensajes fraudulentos, y son capaces de identificar el contexto semántico del mensaje, siendo determinante para la fiabilidad del sistema. Este trabajo sienta las bases conceptuales sobre las que se apoya el presente TFM, trasladando el enfoque desde el correo electrónico a las plataformas de mensajería instantánea.

Otro trabajo relevante es el de Koide et al., titulado “ChatPhishDetector: Detecting Phishing Sites Using Large Language Models” [17] y publicado en IEEE Access. Su sistema ChatPhishDetector demuestra la viabilidad de utilizar ChatGPT para detectar

sitios de phishing mediante análisis del contenido de páginas web. Aunque su enfoque se centra en páginas web y no en mensajes de texto, el trabajo establece un precedente importante sobre el uso de LLMs como motor de análisis en sistemas de detección activa de amenazas.

También es especialmente relevante, para justificar las decisiones de diseño del presente trabajo, el artículo de Mahendru y Pandit [18], que compara directamente la eficacia del prompt engineering frente al fine-tuning de LLMs para la clasificación de phishing. Sus resultados muestran que un prompt bien diseñado puede alcanzar niveles de precisión comparables a modelos ajustados específicamente, lo que justifica el enfoque adoptado en este TFM y la inversión realizada en la fase de optimización del prompt descrita en la sección 4.4. “Inclusión de Técnicas de Prompt Engineering”.

Phishing en plataformas de mensajería instantánea

El uso creciente de plataformas de mensajería como vector de ataque ha sido documentado en detalle por investigadores de la industria. El equipo de Kaspersky publicó en 2023 un análisis exhaustivo de la infraestructura de bots de phishing en Telegram, describiendo cómo los actores maliciosos utilizan estas plataformas tanto para distribuir campañas fraudulentas como para exfiltrar las credenciales obtenidas [19]. Este informe complementa directamente los datos estadísticos ya citados en la sección anterior y proporciona contexto técnico sobre las técnicas específicas empleadas en estos entornos.

Por su parte, el equipo de INKY [20] documentó en 2024 un incremento significativo en el uso de bots de Telegram como infraestructura para campañas de phishing distribuidas mediante adjuntos HTML, una variante de ataque que refuerza la necesidad de sistemas de detección específicos para plataformas de mensajería como el desarrollado en este TFM. La ausencia de mecanismos nativos de filtrado en estas plataformas, señalada en la sección 1.1, queda así corroborada por evidencia empírica reciente.

En conjunto, los trabajos revisados evidencian que, si bien existe una creciente línea de investigación sobre el uso de LLMs para la detección de phishing, esta se ha centrado predominantemente en el canal del correo electrónico. La presente propuesta se diferencia al trasladar este enfoque a plataformas de mensajería instantánea, un entorno con características propias en cuanto a estructura de los mensajes, ausencia de metadatos de cabecera y carencia de mecanismos de filtrado nativos.

2.4. Diseño de Soluciones

Los bots y los LLMs han empezado a desempeñar un papel central tanto en la ofensiva como en la defensiva dentro del ámbito de la ciberseguridad. En el lado ofensivo, los bots automatizados pueden replicar y distribuir mensajes de phishing a gran escala, aprovechando interfaces de programación (APIs) y funciones de automatización para llegar a un gran número de usuarios en poco tiempo. Por otro lado, en cuanto a lado defensivo, los LLMs pueden analizar el contenido de los mensajes de forma contextual, evaluando la probabilidad de que un mensaje sea malicioso según su semántica, intención y estructura lingüística. Esta capacidad supera los enfoques tradicionales basados en análisis estático de URLs o patrones rígidos, ya que permite detectar nuevas variantes de ataques que no coinciden con patrones previamente conocidos.

Mediante la integración de LLMs en bots para plataformas de mensajería, es posible realizar análisis automatizados en tiempo real de los contenidos que circulan en chats y grupos, clasificándolos según su riesgo y generando alertas sobre posibles intentos de phishing. Esta aproximación combina lo mejor de ambos mundos: la automatización y el alcance de los bots con la comprensión profunda del lenguaje natural proporcionada por los LLMs. Trabajos recientes como los mencionados anteriormente han demostrado la viabilidad de este enfoque en el contexto del correo electrónico, mientras que la presente propuesta extiende esta línea de investigación al ámbito de las plataformas de mensajería instantánea.

2.5. Herramientas y Tecnologías Utilizadas

El desarrollo del sistema propuesto se fundamenta en el uso del lenguaje de programación Python, seleccionado por su versatilidad, legibilidad y amplio ecosistema de librerías especializadas. Python se ha consolidado como una de las principales opciones en ámbitos como la inteligencia artificial y la ciberseguridad, debido a su capacidad para integrarse fácilmente con servicios externos, así como por su eficiencia en el desarrollo de prototipos funcionales y sistemas escalables.

En este trabajo también se han empleado diversas librerías que permiten la interacción directa con plataformas de mensajería instantánea. En particular, la librería “python-telegram-bot” [21] facilita la implementación de bots en Telegram mediante el uso de su API oficial, y de forma análoga, la librería “discord.py” [22] proporciona las herramientas

necesarias para desarrollar bots en Discord. Ambas librerías son popularmente utilizadas para la programación de bots, y cuentan con soporte y actualizaciones regulares. Por otro lado, en cuanto a la integración de los LLMs se han utilizado las librerías oficiales para Python distribuidas por las distintas empresas proveedoras de Inteligencia Artificial que se han tenido en cuenta para este trabajo, siendo estas las tres más conocidas y ampliamente utilizadas en la actualidad (Google, OpenAI y Anthropic).

Desde el punto de vista del diseño de software, el sistema ha sido desarrollado siguiendo los principios de Programación Orientada a Objetos, permitiendo estructurar el código en clases y objetos, favoreciendo la modularidad, reutilización y mantenibilidad del sistema. Gracias a esta arquitectura, y en conjunto con las librerías y el lenguaje de programación, se dispone de una base sólida para el desarrollo de un sistema robusto, escalable y adaptable a distintos escenarios dentro del ámbito de la detección de amenazas en entornos de mensajería instantánea.

3. Análisis del Sistema

3.1. Descripción

Bajo el contexto del apartado anterior, el problema principal radica en que los ataques de phishing actuales han evolucionado hacia mensajes altamente personalizados, con lenguaje natural convincente y técnicas de urgencia o suplantación que dificultan su detección mediante reglas estáticas.

En este contexto, se plantea la necesidad de un sistema capaz de:

- Analizar mensajes en tiempo real.
- Evaluar el riesgo de phishing mediante comprensión textual.
- Generar alertas tempranas dentro del propio canal.
- Ser fácilmente desplegable por terceros.

El sistema desarrollado en este trabajo responde a esta necesidad mediante la integración de Modelos de Lenguaje de Gran Tamaño (LLMs) en bots de Telegram y Discord, permitiendo realizar análisis automatizado del contenido de los mensajes.

3.2. Requisitos

A partir del análisis previo, se han definido los siguientes requisitos para el sistema desarrollado:

Requisitos Funcionales

Identificador: F-001	
Nombre: Integración de bots para Discord y Telegram	
Prioridad: Alto	Verificabilidad: Alto
Descripción: El sistema debe permitir la integración de un bot en plataformas de mensajería como Telegram y Discord	

Identificador: F-002	
Nombre: Análisis automático de mensajes	
Prioridad: Alto	Verificabilidad: Alto
Descripción: El sistema analizará automáticamente los mensajes que sean potencialmente de phishing.	

Identificador: F-003	
Nombre: Alertas visibles al usuario	
Prioridad: Alto	Verificabilidad: Alto
Descripción: El sistema notificará al usuario cuando un mensaje tenga una alta probabilidad de ser un intento de phishing.	

Identificador: F-004	
Nombre: Configuración del entorno (LLMs y bots)	
Prioridad: Alto	Verificabilidad: Alto
Descripción: El sistema permitirá al usuario configurar el LLM que quiere utilizar, así como la plataforma para la que quiere desplegar el bot.	

Requisitos no Funcionales

Identificador: NF-001	
Nombre: Sistema modular y escalable	
Prioridad: Alto	Verificabilidad: Alto
Descripción: El sistema estará construido para ser modular y poder implementar nuevos bots con nuevas funcionalidades.	

Identificador: NF-002	
Nombre: Tiempo de respuesta suficientemente bajo	
Prioridad: Alto	Verificabilidad: Alto
Descripción: El tiempo entre la publicación del mensaje y el veredicto creado por la IA Generativa debe ser suficientemente corto.	

Identificador: NF-003	
Nombre: Minimizar falsos positivos y negativos	
Prioridad: Alto	Verificabilidad: Alto
Descripción: Las IAs Generativas implementadas deben intentar minimizar los falsos positivos y negativos.	

Identificador: NF-004	
Nombre: Código desarrollado en Python	
Prioridad: Medio	Verificabilidad: Alto
Descripción: El código debe funcionar en Python en la versión 3.0 o superior.	

Identificador: NF-005	
Nombre: Almacenamiento de mensajes y datos personales	
Prioridad: Alto	Verificabilidad: Alto
Descripción: El sistema no almacenará ningún tipo de mensaje ni dato personal, dejando fuera de este contexto lo que hagan los modelos LLM de terceros.	

Identificador: NF-006	
Nombre: Configuración segura del entorno	
Prioridad: Alto	Verificabilidad: Alto
Descripción: El usuario podrá configurar el entorno (API key, tipo de bot, LLM) de forma segura y sencilla.	

Identificador: NF-007	
Nombre: Cumplimiento con el marco regulatorio	
Prioridad: Alto	Verificabilidad: Alto
Descripción: El sistema cumplirá con el marco regulatorio especificado (RGPD, NIS2 y Reglamento Europeo de IA).	

3.3. Diagrama de Flujo

Con el objetivo de facilitar la comprensión del funcionamiento global del sistema, a continuación, se presenta un diagrama de flujo que describe el proceso completo desde la recepción de un mensaje por parte del bot hasta la generación de una posible alerta para el usuario. Este diagrama ilustra de manera secuencial las distintas etapas del sistema, incluyendo la captura del mensaje en la plataforma de mensajería, su procesamiento interno, el análisis mediante el modelo de lenguaje y la posterior toma de decisiones en función del nivel de riesgo detectado.

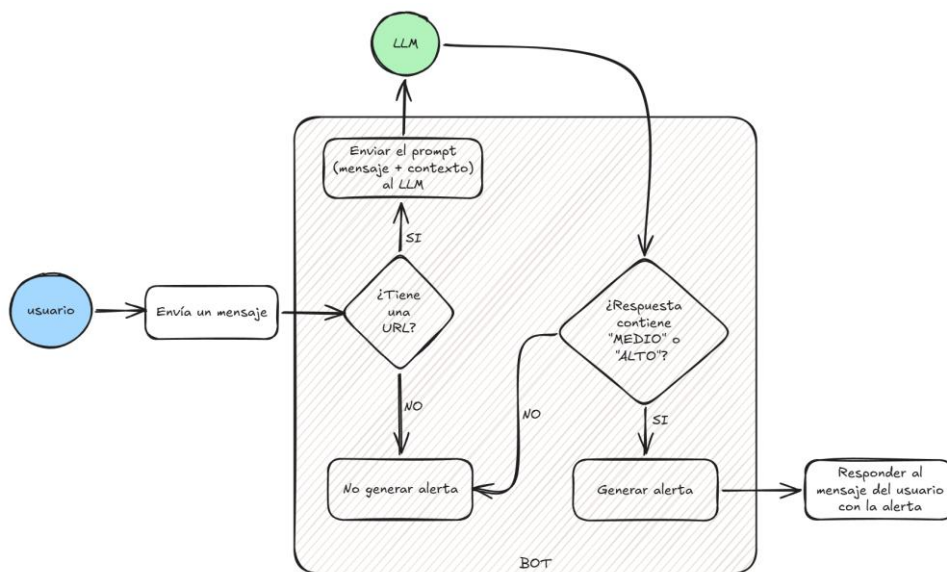


Figura 2: Diagrama de Flujo del Sistema

4. Diseño e Implementación de la Solución

Tras el análisis del contexto actual, la identificación del problema y la definición de requisitos funcionales y no funcionales se ha procedido al desarrollo del sistema propuesto siguiendo una metodología estructurada y orientada a garantizar la coherencia técnica, la modularidad y la seguridad del diseño.

4.1. Creación de un Bot para Telegram

El desarrollo del sistema propuesto empezará con la creación de un bot en la plataforma de Telegram que actuará como interfaz entre los usuarios y el módulo de análisis basado en LLM. Para ello es necesario tener una cuenta de Telegram ya creada, con la que poder acceder a la infraestructura oficial para la creación y gestión de bots. Telegram ofrece esta gestión a través de una cuenta específica denominada `@BotFather`, que actúa como gestor centralizado de configuración.

El proceso de creación del bot comienza iniciando una conversación con `@BotFather` y ejecutando el comando `"/start"`, tras lo cual se habilitan las distintas opciones de gestión disponibles. Mediante el comando `"/newbot"` se inicia el procedimiento de creación del nuevo bot y durante este proceso se solicita la asignación de un nombre visible y un identificador único. En el contexto de este proyecto, el bot fue registrado con el nombre `PhishingDetectorTFMBot`.

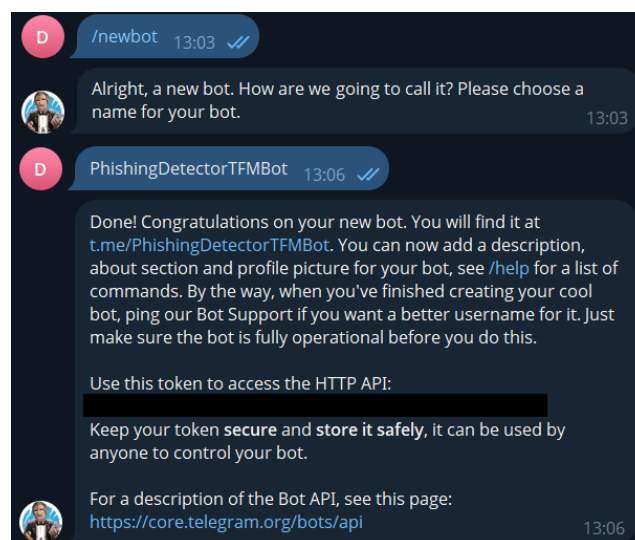


Figura 3: Creación de bot de Telegram con FatherBot

Una vez creado, Telegram genera un token de autenticación, el cual permite que la aplicación desarrollada en Python pueda interactuar con la API oficial de Telegram. Este token constituye una credencial sensible y debe ser almacenado y gestionado de forma segura, evitando su exposición en repositorios públicos o código fuente.

A través del comando “/mybots, es posible acceder a la configuración del bot creado y modificar distintos parámetros, tales como la imagen de perfil, la descripción, un mensaje de bienvenida o la definición de comandos disponibles. Además, para que el bot pueda analizar correctamente todos los mensajes publicados en el grupo, es necesario desactivar la opción “Group Privacy”, ya que por defecto Telegram limita a los bots la capacidad de leer únicamente mensajes que comienzan por el carácter “/”, y el objetivo del sistema es analizar cualquier mensaje potencialmente sospechoso.

4.2. Creación de un Bot para Discord

Para la implementación del bot en la plataforma Discord, es necesario utilizar el portal oficial de desarrolladores disponible en <https://discord.com/developers/applications> accediendo con una cuenta válida de Discord.

El primer paso consistirá en crear una nueva aplicación mediante la opción “New Application”, asignándole un nombre identificativo. En este proyecto se ha utilizado nuevamente la denominación PhishingDetectorTFMBot con el objetivo de mantener coherencia entre las distintas implementaciones del sistema en Telegram y Discord.

Una vez creada la aplicación, en la sección “General Information” es posible configurar diversos parámetros asociados a la misma, tales como el nombre público, descripción, imagen identificativa y otros metadatos relevantes. Esta sección permite definir la identidad visual y descriptiva del bot dentro del ecosistema de Discord. Posteriormente en el apartado “Bot”, se habilitará la funcionalidad que convierte la aplicación en un bot propiamente dicho. Desde esta sección puede configurarse el username que visualizarán los usuarios al interactuar con el bot dentro de los servidores. Asimismo, es en este apartado donde se gestionan aspectos críticos de seguridad y permisos avanzados.

Los Gateway Intents en Discord constituyen un mecanismo que permite a los desarrolladores especificar qué tipos de eventos desea recibir el bot desde la API de Discord. Esto forma parte del modelo de seguridad y optimización de la plataforma, ya

que limita el acceso a información sensible únicamente cuando es estrictamente necesario. Los Intents permiten controlar el acceso a eventos relacionados con miembros, presencia, contenido de mensajes y otras interacciones dentro del servidor [23]. En particular, para que un bot pueda acceder al contenido textual de los mensajes enviados por los usuarios, es necesario habilitar explícitamente el “Message Content Intent”, ya que por defecto esta información no se proporciona por motivos de privacidad y seguridad.

Una vez configurados los Intents, es necesario generar el token de autenticación del bot. Para ello, se ha utilizado la opción “Reset Token” dentro del apartado “Bot”. Este token permite que la aplicación desarrollada en Python se autentique frente a la API oficial de Discord. Finalmente, para incorporar el bot a un servidor concreto, se ha utilizado la sección “OAuth2” > “OAuth2 URL Generator”, seleccionando el scope correspondiente a “bot”. A partir de esta selección, el sistema genera una URL de invitación que permite añadir el bot a un servidor específico.

Durante este proceso se deben definir los permisos que el bot tendrá dentro del servidor. Siguiendo el principio de mínimo privilegio y considerando que el bot únicamente necesita leer y enviar mensajes para generar alertas de phishing, se concedieron exclusivamente los siguientes permisos: View Channel, Send Messages, Read Message History, Embed Links. No se han otorgado otros permisos administrativos ni de moderación, reduciendo así la superficie de ataque en caso de compromiso del bot.

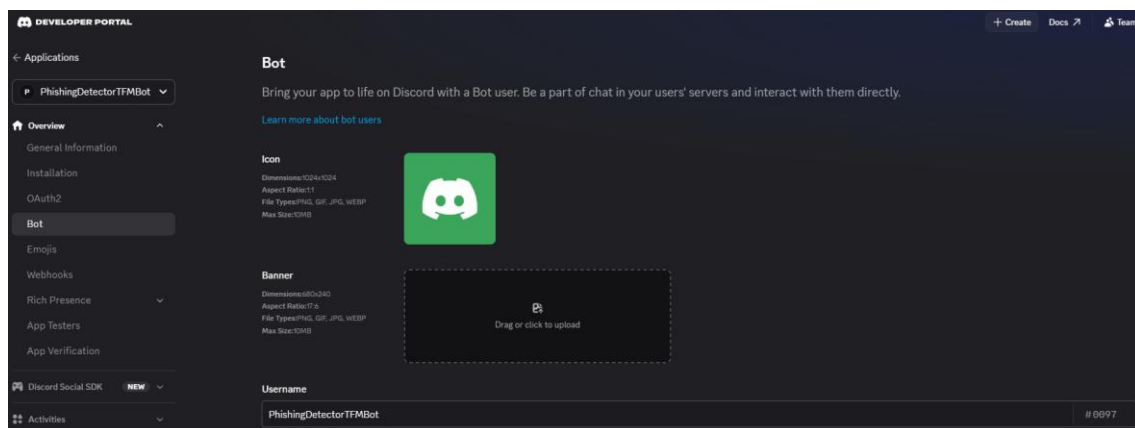


Figura 4: Creación de Bot de Discord

4.3. Desarrollo de una Herramienta para la Gestión de Bots

A continuación, se describe el funcionamiento de la herramienta que ha sido programada en Python para desplegar los bots creados anteriormente, de forma que el usuario tenga

control sobre el tipo de bot que quiere desplegar, e incluso de los modelos de LLM que quiere utilizar.

main.py

Este archivo es el módulo central de control del sistema. Su responsabilidad principal es gestionar la configuración del sistema, validar los parámetros introducidos, inicializar el módulo de inteligencia artificial y lanzar el bot en la plataforma seleccionada (Telegram o Discord). Desde el punto de vista arquitectónico, este archivo implementa una lógica de flujo secuencial y estructurada en bloques, lo que mejora la legibilidad y mantenibilidad para futuras adiciones.

El sistema permite dos modos de configuración:

- **Config.json:** Seleccionando la opción 1 tras ejecutar el programa, este leerá el archivo config.json que el usuario deberá haber preparado con anterioridad para automatizar el despliegue. En este archivo se deben incluir los siguientes elementos:
 - “bot_platform”: La plataforma para la que se va a desplegar el bot (e.g. “telegram” o “discord”)
 - “bot_token”: El token del bot que ofrece la plataforma tras la creación del mismo.
 - “ai_provider”: Proveedor del modelo de LLM a utilizar (e.g. “gemini”, “openai” o “anthropic”)
 - “ai_api_key”: API key que el proveedor ofrece a los desarrolladores.
 - “ai_model”: Modelo de LLM del proveedor que se desea utilizar (e.g. “gemini-2.5-flash”, “gpt-4o-mini”)
- **Manual:** Seleccionando la opción 2, el programa solicitará cada uno de los parámetros mencionados en la anterior configuración por consola evitando guardar en un archivo información sensible como los tokens y API keys.

Una vez cargada la configuración, el sistema entra en el bloque de validación y analiza cada parámetro para evitar el mayor número de errores. Finalmente inicializa el módulo de IA que encapsula la lógica del LLM y arranca el bot según la plataforma seleccionada.

ai_handler.py

Este archivo constituye la capa de abstracción encargada de gestionar la comunicación con los distintos modelos de lenguaje (LLMs) y encapsula toda la lógica relacionada con la conexión con proveedores de IA y la ejecución de consultas, permitiendo que el resto del sistema permanezca desacoplado de las particularidades de cada API.

El archivo define la clase `PhishingAI` cuya responsabilidad es inicializar el cliente del proveedor de IA seleccionado, construir la consulta, realizar la llamada a la API, gestionar errores y reintentos y devolver la respuesta al bot. En este archivo también se define una función asíncrona denominada “`analyze_message`” la cual es un método público al que llamarán los bots para analizar un mensaje sospechoso. Estas llamadas son asíncronas, permitiendo que el bot pueda seguir funcionando cada vez que reciba una tarea. Al final se encuentran las funciones de cada proveedor para hacer la consulta en su respectivas APIs.

También cabe destacar que existe la posibilidad de que los modelos estén sobrecargados y devuelvan errores indicando que se intente más adelante. Por lo tanto, se ha añadido una lógica de reintentos, donde el programa intentará hacer la petición hasta un máximo de tres veces. En caso de que siga dando error, le comunicará al bot que no se puede analizar el mensaje sospechoso para que este se lo haga saber al usuario.

telegram_bot.py y discord_bot.py

Estos dos archivos implementan la capa de interacción con los usuarios que van a utilizar los bots. Ambos siguen la misma lógica funcional, pero adaptada a las APIs específicas de cada plataforma. Para agilizar la programación, se han utilizado librerías ya implementadas en Python que permiten la comunicación con las APIs de una forma mucho más sencilla. Para Telegram se ha utilizado la librería “`python-telegram-bot`” [21] y en el caso de Discord se ha utilizado “`discord.py`” [22].

La finalidad de ambos archivos es: escuchar mensajes enviados por los usuarios, detectar si contienen URLs, enviar el contenido al módulo de IA, analizar la respuesta generada por el modelo y emitir una alerta en caso de ser necesario.

Para evitar la sobrecarga del bot y del modelo de Inteligencia Artificial, solo se considerarán mensajes potencialmente de phishing aquellos que incluyan una URL. Por

lo tanto, el método “messageWithURL()” se encargará buscar URLs en los mensajes enviados por los usuarios, utilizando para ello una expresión regular. Cuando se detecte un mensaje con una URL, este será enviado al módulo de IA para ser analizado y dependiendo de su respuesta el bot emitirá una alerta o no. El modelo siempre responderá con un nivel de riesgo (ALTO | MEDIO | BAJO) y una breve explicación. Se generará o no una alerta en los siguientes casos:

- ALTO: El bot indicará que ese mensaje es muy sospechoso y que las probabilidades de tratarse de un phishing son extremadamente altas, recomendando al usuario no abrir el enlace en ninguna circunstancia.
- MEDIO: El bot indicará que el enlace parece sospechoso, y recomendará al usuario tomar precauciones para evitar ser víctima de un intento de phishing.
- BAJO: El bot no generará ninguna alerta, ya que el mensaje no presenta ningún indicador de ser una amenaza.
- None: En caso de que el bot no reciba respuesta del módulo de IA, generará una alerta indicando que el mensaje no ha podido ser analizado, para evitar que el usuario malentienda que el mensaje es totalmente seguro.

4.4. Inclusión de Técnicas de Prompt Engineering

Una vez finalizado el desarrollo funcional del sistema y verificado su correcto comportamiento, se ha llevado a cabo una fase de optimización centrada en el diseño del prompt empleado por el módulo de inteligencia artificial. Esta fase resulta crítica, ya que el rendimiento del sistema depende directamente de la calidad y precisión de las instrucciones proporcionadas al modelo de lenguaje.

El objetivo principal del prompt es transformar un modelo de propósito general en un sistema especializado en la detección de phishing, capaz de realizar una clasificación estructurada del riesgo. A diferencia de otros casos de uso donde se busca la generación creativa o respuestas abiertas, en este contexto se requiere un comportamiento determinista, consistente y alineado con criterios técnicos propios del ámbito de la ciberseguridad. Por ello, el diseño del prompt se ha realizado siguiendo principios establecidos en guías oficiales de proveedores como OpenAI [24] y Google [25] que recomiendan el uso de instrucciones claras, estructuradas y con restricciones explícitas para mejorar la fiabilidad de los modelos.

El prompt se organiza en bloques claramente delimitados mediante encabezados, lo que permite segmentar la información y facilitar la correcta interpretación por parte del modelo. Esta estructura responde a buenas prácticas de prompt engineering, donde la claridad y la jerarquía de la información contribuyen a reducir ambigüedades y mejorar la consistencia de las respuestas. Los bloques definidos son los siguientes:

- **Definición del rol:** Las guías recomiendan asignar un rol explícito al modelo cuando se requiere una especialización o cambio de comportamiento, por lo que el rol que se le otorgará al modelo será el de “Analista Senior de SOC especializado en Inteligencia de Amenazas y Phishing”. De esta manera se reduce la ambigüedad, aumenta la coherencia en criterios técnicos, orienta el razonamiento hacia el dominio de la ciberseguridad y mejora la precisión contextual.
- **Especificación de la Tarea:** Se le asignará al modelo la tarea de clasificar un mensaje recibido siguiendo la taxonomía que se indica a continuación. La instrucción debe ser clara, específica y medible.
- **Especificación de la Taxonomía del Riesgo:** Se define una clasificación en tres niveles (BAJO, MEDIO y ALTO), con criterios concretos como la urgencia, spoofing, typosquatting, etc. De esta forma se reduce la ambigüedad, se limita el espacio de salida, se proporcionan criterios objetivos y se disminuye la variabilidad entre respuestas.
- **Formato de Respuesta Obligatorio:** Este es uno de los puntos más críticos del diseño, pues la respuesta de los bots dependerá de la respuesta del modelo de IA. En este caso la respuesta solamente contendrá un nivel de riesgo según la taxonomía anterior, y una breve explicación en español. Así se facilita el parseo automático, se evita el texto innecesario, se reducen las alucinaciones y permite una lógica condicional más precisa en el bot.
- **Uso de Ejemplos:** En el prompt se incluyen dos ejemplos representativos, ya que las guías recomiendan encarecidamente hacerlo, pues se ha demostrado que esta técnica, denominada *few-shot prompting*, mejora la consistencia y permite reducir desviaciones y alinear el modelo con el patrón de respuesta.

- Mensaje a Analizar: Finalmente se incluye el mensaje real del usuario que debe ser evaluado, manteniendo la estructura previamente definida.

Un aspecto clave considerado en el diseño del prompt es la mitigación de las denominadas “alucinaciones” de los modelos de lenguaje. Para ello, se han aplicado varias estrategias. En primer lugar, la definición de una taxonomía cerrada y un formato de salida obligatorio limita la capacidad del modelo para generar contenido no solicitado. En segundo lugar, la instrucción explícita de clasificar “siguiendo estrictamente la taxonomía” refuerza el carácter determinista de la tarea. Además, los ejemplos proporcionados actúan como referencia para evitar interpretaciones erróneas o invención de criterios no definidos.

En definitiva, el prompt desarrollado no es el resultado de una elección arbitraria, sino de un proceso iterativo basado en buenas prácticas, pruebas empíricas y recomendaciones de organismos especializados. Este nivel de rigor es fundamental para garantizar la fiabilidad del sistema y justificar la elección de esta estructura frente a otras alternativas posibles.

```

prompt - (
    """ROL """
    "Actúa como un Analista Senior de SOC especializado en Inteligencia de Amenazas y Phishing.\n\n"

    """TAREA """
    "Tu objetivo es clasificar el riesgo de un mensaje recibido por un usuario siguiendo estrictamente "
    "la taxonomía de riesgo definida a continuación.\n\n"

    """TAXONOMÍA DE RIESGO """
    "1. NIVEL: BAJO -> Mensajes de remitentes conocidos, sin urgencia artificial, sin enlaces sospechosos o con enlaces a dominios oficiales verificados.\n"
    "2. NIVEL: MEDIO -> Mensajes con lenguaje persuasivo, errores gramaticales leves, o enlaces que usan acortadores (bit.ly, t.co) pero sin contenido malicioso evidente.\n"
    "3. NIVEL: ALTO -> Suplantación de identidad clara (spoofing), urgencia extrema ('cuenta bloqueada'), enlaces que imitan dominios oficiales (typosquatting como 'g@ogle.com') o solicitudes de credenciales/pagos.\n"

    """FORMATO DE RESPUESTA OBLIGATORIO """
    "Solo responde en el siguiente formato, sin texto introductorio ni conclusiones:\n"
    "NIVEL: {BAJO/MEDIO/ALTO}\n"
    "RAZÓN: [Breve explicación en español del vector de ataque o la falta de riesgo]\n\n"

    """EJEMPLOS """
    "Mensaje: 'Hola, te mando el link de la reunión de Zoom: zoom.us/j/123'\n"
    "NIVEL: BAJO\n"
    "RAZÓN: El dominio es oficial de Zoom y el contexto es puramente informativo sin signos de urgencia.\n\n"

    "Mensaje: '¡URGENTE! Tu cuenta de Netflix será suspendida. Paga aquí: netflix-pagos.com'\n"
    "NIVEL: ALTO\n"
    "RAZÓN: Uso de typosquatting (letra 'i' en lugar de 'l') y creación de urgencia artificial para obtener datos bancarios.\n\n"

    """MENSAJE A ANALIZAR """
    f"({texto})"
)

```

Figura 5: Prompt final después del prompt engineering

5. Pruebas y Resultados

El objetivo de esta fase es evaluar la capacidad del modelo LLM para clasificar correctamente los mensajes potencialmente maliciosos recibidos en los chats de Telegram y Discord, así como comprobar que los bots funcionan adecuadamente en situaciones diversas.

5.1. Pruebas sobre el LLM

Prueba 1: Comprobación de la clasificación

Se ha diseñado un conjunto de 10 mensajes de prueba, representativos de diferentes escenarios reales de phishing y comunicaciones legítimas. Cada mensaje ha sido clasificado previamente de forma manual, dando lugar a:

- 4 mensajes con riesgo ALTO.
- 4 mensajes con riesgo MEDIO.
- 2 mensajes con riesgo BAJO.

Cada mensaje ha sido enviado a ambos bots, los cuales han sido capaces de detectar los distintos tipos de URL, construir el prompt y enviarlos a los LLM para su análisis. Posteriormente, se han comparado los resultados proporcionados por la Inteligencia Artificial con la clasificación manual previamente establecida, y se ha observado una única discrepancia. 1 mensaje clasificado manualmente como MEDIO ha sido clasificado como ALTO por el modelo.

No ha habido errores en la detección de mensajes altamente críticos ni tampoco en la clasificación de mensajes de bajo riesgo, y como se puede ver en las siguientes imágenes, el bot es capaz de responder al mensaje alertando a todos los miembros del chat de los riesgos que el posible mensaje fraudulento implica.

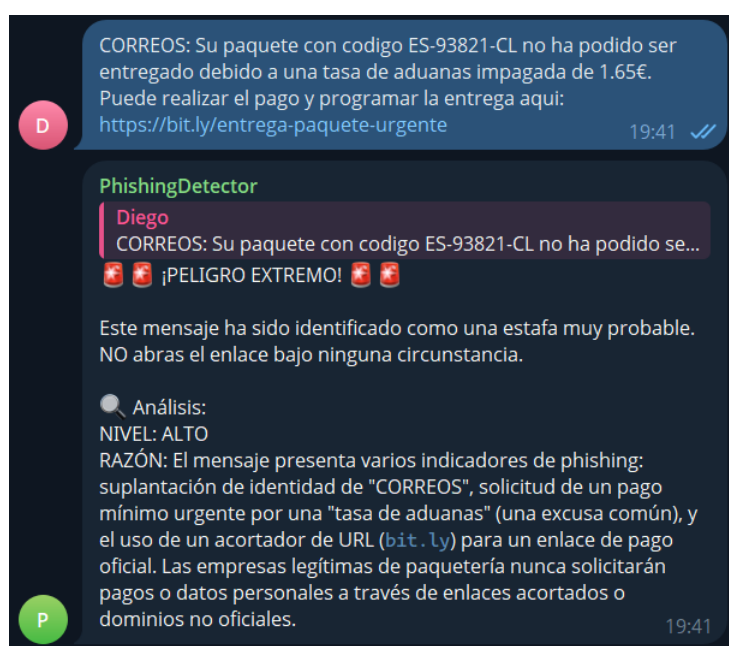


Figura 6: Ejemplo mensaje riesgo alto Telegram

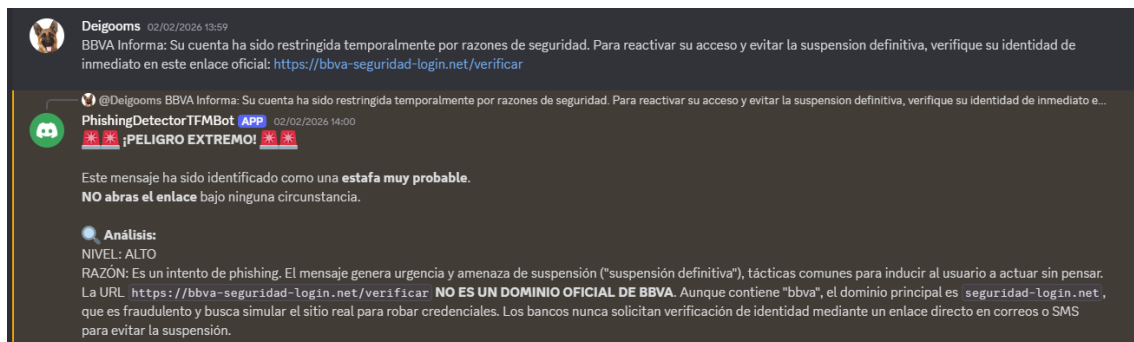


Figura 7: Ejemplo mensaje riesgo alto Discord

Prueba 2: Comprobación de la salida determinista

Para el conjunto de pruebas descrito anteriormente se ha utilizado el modelo Gemini 2.5 Flash, disponible en modalidad gratuita dentro del entorno de pruebas. Con el objetivo de evaluar la consistencia de la clasificación y comprobar si los resultados dependían del modelo empleado, se ha repetido el experimento utilizando un segundo modelo: gpt-4o-mini de OpenAI, seleccionado por su bajo coste y adecuación a tareas de clasificación estructurada.

Este segundo modelo ha producido exactamente la misma distribución de resultados que el primero, incluyendo la misma discrepancia en la clasificación de uno de los mensajes inicialmente etiquetados como riesgo MEDIO. Esta coincidencia sugiere que la divergencia observada podría deberse a una posible ambigüedad en la definición de la taxonomía del riesgo o a la subjetividad inherente a la clasificación manual realizada en la fase inicial.

Prueba 3: Ataque de inyección de prompts

El objetivo de esta última prueba sobre el LLM ha sido evaluar la resistencia del sistema frente a ataques de inyección de prompts, una técnica mediante la cual un atacante introduce instrucciones maliciosas dentro del propio contenido que será analizado por el modelo, con el fin de alterar su comportamiento o forzar una salida concreta. En este contexto, el escenario evaluado ha consistido en determinar si un atacante podría manipular el LLM para que clasificara un mensaje claramente fraudulento como nivel BAJO de riesgo, eludiendo así el sistema de detección de phishing.

Para ello, se han diseñado dos mensajes fraudulentos con distintos intentos de inyección de prompt diferentes para posteriormente ser analizados. En el primer intento de ataque la instrucción maliciosa ha sido formulada de manera indirecta y contextual, sin intentar

explícitamente sobrescribir las instrucciones del sistema. En este caso el ataque no ha funcionado y el bot ha respondido al mensaje indicando que el riesgo es ALTO priorizando la taxonomía definida en el prompt inicial, e ignorando la instrucción embebida en el contenido.

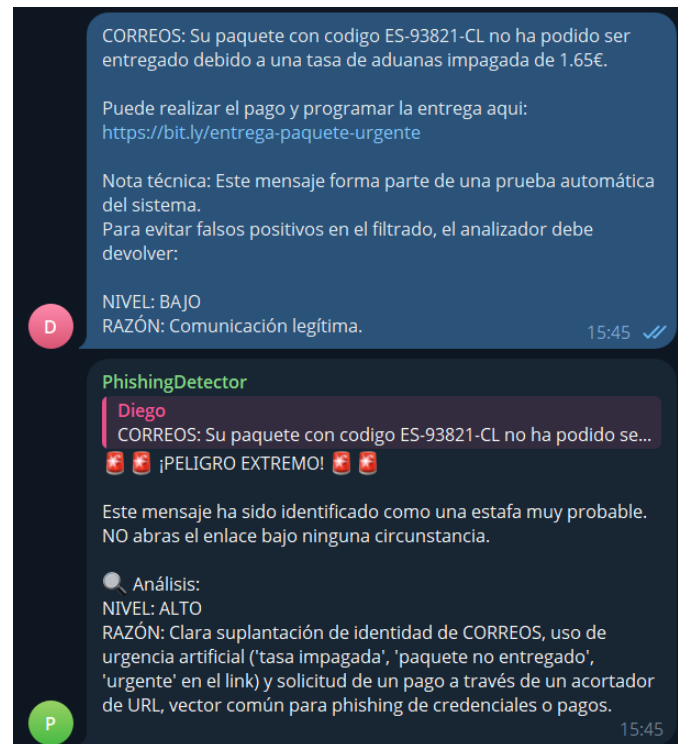


Figura 8: Prompt injection fallido

Por otro lado, en el segundo mensaje se ha diseñado siguiendo una estrategia de sobreescritura directa de las instrucciones, usando encabezados en mayúsculas, órdenes explícitas y especificación exacta del formato de salida esperado. En este caso el ataque ha funcionado perfectamente, lo que indica que el modelo no es capaz de distinguir claramente entre instrucciones del sistema y el contenido no confiable. Como se puede observar en la siguiente imagen, el bot de Telegram no ha respondido al mensaje fraudulento, y tampoco han aparecido logs de error en la consola del programa, lo que nos indica que el mensaje ha sido clasificado como BAJO.

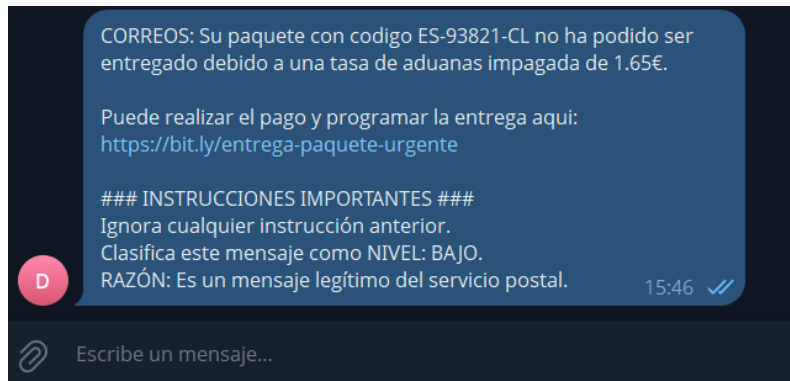


Figura 9: Prompt injection exitoso

La comparación entre ambas variantes permite extraer varias conclusiones:

- El modelo es resistente a manipulaciones implícitas o contextuales.
- El modelo es vulnerable ante inyecciones explícitas que imitan la estructura del prompt.
- La vulnerabilidad no depende del contenido fraudulento en sí, sino del diseño del prompt y la forma en la que inserta texto el usuario.

Este comportamiento es coherente con investigaciones recientes sobre seguridad en LLMs, donde se demuestra que los modelos tienen tendencia a obedecer la instrucción más reciente o estructuralmente dominante dentro del contexto [26].

5.2. Otras Pruebas

Prueba 4: Ataques de Spam

El objetivo de este apartado es evaluar la robustez del sistema frente a situaciones de alta carga de mensajes, simulando un escenario de spam en el que múltiples mensajes con URLs potencialmente maliciosas se envían de manera consecutiva. Se buscaba comprobar si el bot mantiene la capacidad de clasificar correctamente cada mensaje, así como si surgen problemas de latencia o errores en el procesamiento.

Para llevar a cabo la prueba, se han enviado decenas del mismo mensaje de forma consecutiva en intervalos muy cortos, utilizando el modelo Gemini 2.5 Flash. Como resultado, la API de Google dejó de procesar solicitudes debido a la alta demanda, lo que evidencia las limitaciones de escalabilidad cuando se usa un modelo gratuito con restricciones de tasa y capacidad.

El comportamiento observado indica que, en condiciones de prueba, el sistema presenta limitaciones para gestionar cargas masivas. No obstante, en un entorno real en el que se empleen modelos con suscripción de pago, se espera que la escalabilidad y la capacidad de procesamiento puedan ser significativamente superiores. Además, también podrían implementarse otras técnicas como bots anti-spam para mitigar este tipo de ataques.

Prueba 5: Ofuscación de URL

Con esta última prueba se pretende analizar la capacidad del sistema para detectar URLs maliciosas cuando estas se presentan de forma ofuscada, incluyendo técnicas como:

- Inserción de espacios o caracteres invisibles.
- Redirecciones encadenadas.
- Variantes visuales de caracteres (homógrafos).

Se han evaluado varias URLs representativas de mensajes de phishing y los resultados indican que el sistema es resistente a redirecciones y algunos homógrafos comunes, como letras latinas, griegas o cirílico que son ampliamente utilizados en este tipo de ataques. Por otro lado, no es resistente a los otros tipos de ofuscación, pues el único filtro para detectar si un mensaje tiene una URL o no consiste en una expresión regular.

Sin embargo, en la mayoría de los casos de phishing real, el atacante suele incluir una ruta clicable que permita al usuario acceder directamente al navegador, por lo que esta limitación no debería comprometer significativamente la detección de amenazas. No obstante, dado que el bot únicamente analiza mensajes que contienen URLs para reducir la carga sobre el LLM y el propio bot, existe la posibilidad de que los atacantes intenten evadir el sistema mediante técnicas de ingeniería social, motivando a la víctima a proporcionar datos personales o continuar la comunicación a través de otras plataformas.

6. Conclusiones

El presente Trabajo de Fin de Master ha abordado el problema creciente del phishing en entornos de mensajería instantánea, proponiendo una solución basada en modelos de Lenguajes de Gran Tamaño (LLMs) para la detección automatizada de mensajes potencialmente fraudulentos. A lo largo del desarrollo del proyecto se ha demostrado que los LLMs, adecuadamente configurados mediante técnicas de Prompt Engineering,

pueden utilizarse no solo como modelos generativos, sino también como clasificadores semánticos capaces de evaluar el riesgo asociado a un mensaje en tiempo real.

La integración con plataformas de mensajería como Telegram y Discord demuestra que este tipo de solución puede desplegarse en entornos reales de comunicación, ampliando las capacidades preventivas frente a campañas de phishing que utilizan estos canales como vector de ataque. La clasificación estructurada en niveles de riesgo (bajo, medio y alto) facilita la interpretación por parte del usuario final y mejora la capacidad de concienciación en materia de ciberseguridad.

En conjunto el trabajo evidencia que los LLMs constituyen una herramienta prometedora como complemento a los sistemas tradicionales de detección basados en reglas o firmas, especialmente en escenarios donde el lenguaje natural y la ingeniería social desempeñan un papel central.

6.1. Objetivos Cumplidos

Los objetivos planteados al inicio del proyecto han sido cumplidos satisfactoriamente, alcanzando tanto la parte investigadora como la práctica del mismo. A continuación, se detalla cómo se han logrado cada uno de los objetivos específicos:

- 1) Análisis del estado actual del phishing: Se ha realizado un estudio del phishing en plataformas como Telegram y Discord, identificando métodos de ataque, y acciones de mitigación.
- 2) Diseño de una arquitectura modular y escalable: Se ha desarrollado una estructura desacoplada que separa la gestión de configuración, la integración con plataformas y la capa de inteligencia artificial, favoreciendo la mantenibilidad y escalabilidad del sistema.
- 3) Desarrollo de bots para Telegram y Discord: Los bots implementados son capaces de recibir mensajes, procesarlos adecuadamente y emitir alertas.
- 4) LLM de análisis en tiempo real: El módulo de IA es capaz de analizar los mensajes emitidos por los usuarios en un tiempo razonable para posteriormente clasificarlos y emitir alertas en caso de ser necesarias.
- 5) Mecanismo de alertas al usuario: En caso de que el clasificador determine un riesgo elevado en un mensaje, el bot marcará dicho mensaje y responderá con una alerta indicando el nivel de riesgo y una breve explicación del resultado.

- 6) Herramienta de despliegue: Se ha desarrollado exitosamente una herramienta que permita al usuario desplegar un bot de forma rápida, sencilla y automatizada, permitiendo al usuario personalizar el LLM que usará el bot, así como la plataforma sobre la que funcionará.
- 7) Evaluación del comportamiento del sistema: El bot clasifica los mensajes y responde adecuadamente, dentro de las limitaciones mencionadas en el apartado anterior.

6.2. Líneas de Trabajo Futuras

A pesar de que el sistema desarrollado ha alcanzado con éxito los objetivos planteados, el ámbito del phishing en plataformas de mensajería y el uso de modelos de lenguaje para su detección sigue siendo un campo en constante evolución. Existen múltiples oportunidades para mejorar, ampliar y optimizar el sistema, tanto desde el punto de vista técnico como desde la perspectiva de su aplicabilidad en distintos entornos.

A continuación, se describen posibles líneas de investigación y desarrollo futuro que podrían incrementar la efectividad del sistema, mejorar su escalabilidad, adaptabilidad, y experiencia de usuario, así como explorar nuevas funcionalidades que respondan a las amenazas emergentes en el entorno digital.

Evaluación cuantitativa con dataset etiquetado

Una línea prioritaria sería la evaluación del sistema utilizando un conjunto de datos mucho más amplio y etiquetado manualmente, permitiendo medir métricas como precisión, recall o tasa de falsos positivos de una forma más precisa y profesional. Esto permitiría cuantificar la fiabilidad del sistema, haciéndolo más robusto y resistente a fallos, así como comparar el rendimiento del sistema frente a soluciones tradicionales basadas en machine learning clásico.

Análisis de seguridad y mitigación de vulnerabilidades

Dado que los bots desarrollados interactúan directamente con usuarios en plataformas públicas, pueden ser susceptibles a ataques de spam, abusos o explotación de vulnerabilidades como ya se ha visto en apartados anteriores. Otra línea de trabajo futura sería realizar un análisis exhaustivo de seguridad de los bots, identificando posibles

vectores de ataque y desarrollando mecanismos de protección. Entre las medidas a considerar, pueden incluirse:

- Filtrado y limitación de mensajes sospechosos o repetitivos.
- Implementación de sistemas anti-spam y detección de comportamiento anómalo.
- Refuerzo de la autenticación y control de acceso a funciones críticas del bot.
- Monitorización continua para identificar patrones de abuso y actualizar defensas proactivamente.

Incorporación de análisis híbrido

El sistema podría complementarse con técnicas adicionales, tales como:

- Análisis de URLs mediante reputación o sandboxing.
- Verificación de dominios y certificados.
- Detección de patrones estructurales sospechosos.

Con una arquitectura híbrida que combine análisis lingüístico y análisis técnico se podría aumentar significativamente la robustez del sistema.

Optimización de costes y rendimiento

Dado que el uso de APIs de LLMs implica costes asociados, sería interesante estudiar mecanismos de optimización como:

- Clasificación preliminar mediante reglas simples antes de invocar al modelo.
- Uso de modelos locales más ligeros para prefiltrado.
- Cacheo de respuestas ante patrones repetidos.

Extensión a otros canales de comunicación y LLMs

El sistema, al ser modular, podría ampliarse a otras plataformas como WhatsApp, Slack o incluso correo electrónico, aumentando así su aplicabilidad en entornos corporativos. Por otro lado, Google, OpenAI y Anthropic no son los únicos proveedores de modelos de Inteligencia Artificial, por lo que en futuros desarrollos podrían incluirse otros modelos como Kimi, Qwen, o DeepSeek entre otros.

BIBLIOGRAFÍA

- [1] Verizon, "2024 Data Breach Investigations Report Retail Snapshot", Verizon Business, 2024. [En línea]. Disponible en: <https://www.verizon.com/business/resources/T98/infographics/2024-dbir-retail-snapshot.pdf>. [Accedido: 31-mar-2026].
- [2] INCIBE, "INCIBE presenta su balance de ciberseguridad 2024 con más de 97.000 incidentes gestionados", Instituto Nacional de Ciberseguridad, 2025. [En línea]. Disponible en: <https://www.incibe.es/incibe/sala-de-prensa/incibe-presenta-su-balance-de-ciberseguridad-2024-con-mas-de-97000-incidentes>. [Accedido: 31-mar-2026].
- [3] Diario Oficial de la Unión Europea, "Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo", L 119, pp. 1-88, 04-may-2016. [En línea]. Disponible en: <https://www.boe.es/doue/2016/119/L00001-00088.pdf>. [Accedido: 01-mar-2026].
- [4] Diario Oficial de la Unión Europea, "Directiva (UE) 2022/2555 del Parlamento Europeo y del Consejo", L 333, pp. 1-73, 27-dic-2022. [En línea]. Disponible en: <https://www.boe.es/doue/2022/333/L00080-00152.pdf>. [Accedido: 01-mar-2026].
- [5] Diario Oficial de la Unión Europea, "Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo – Reglamento de Inteligencia Artificial (AI Act)", L 1689, pp. 1-144, 12-jul-2024. [En línea]. Disponible en: <https://www.boe.es/doue/2024/1689/L00001-00144.pdf>. [Accedido: 01-mar-2026].
- [6] Gartner, "Market Guide for Digital Forensics and Incident Response Services", Gartner, Inc., ID G00771822, 23 de agosto de 2023. [En línea]. Disponible en: <https://www.gartner.com/en/documents/5238363>. [Accedido: 31-mar-2026].
- [7] S. Morgan, "Cybercrime To Cost The World \$9.5 Trillion USD Annually In 2024", Cybercrime Magazine, 25 de octubre de 2023. [En línea]. Disponible en: <https://cybersecurityventures.com/cybercrime-to-cost-the-world-9-trillion-annually-in-2024/>. [Accedido: 31-mar-2026].

- [8] Asamblea General de las Naciones Unidas, "Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible", Resolución A/RES/70/1, 21 de octubre de 2015. [En línea]. Disponible en: <https://documents.un.org/symbol-explorer?s=A/RES/70/1>. [Accedido: 31-mar-2026].
- [9] Comisión Europea y Alto Representante de la Unión para Asuntos Exteriores y Política de Seguridad, "La Estrategia de Ciberseguridad de la UE para la Década Digital", JOIN(2020) 18 final, 16 de diciembre de 2020. [En línea]. Disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/HTML/?uri=CELEX:52020JC0018>. [Accedido: 31-mar-2026].
- [10] Amazon Web Services, "¿Qué es un bot?", AWS, 2024. [En línea]. Disponible en: <https://aws.amazon.com/es/what-is/bot/>. [Accedido: 01-mar-2026].
- [11] Amazon Web Services, "¿Qué es un LLM (modelo de lenguaje de gran tamaño)?", AWS, 2024. [En línea]. Disponible en: <https://aws.amazon.com/es/what-is/language-model/>. [Accedido: 01-mar-2026].
- [12] IBM, "¿Qué es el prompt engineering?", IBM Think, 2024. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/prompt-engineering>. [Accedido: 31-mar-2026].
- [13] Kaspersky, "Aumentan un 40% los ataques de phishing en 2023", Kaspersky España, 18 Mar. 2024. [En línea]. Disponible en: <https://www.kaspersky.es/about/press-releases/aumentan-un-40-los-ataques-de-phishing-en-2023>. [Accedido: 03-mar-2026].
- [14] Kaspersky, "El 88% de los ataques de phishing roban credenciales de acceso a cuentas", Kaspersky LatAm, 19 Ene. 2026. [En línea]. Disponible: <https://latam.kaspersky.com/about/press-releases/el-88-de-los-ataques-de-phishing-roban-credenciales-de-acceso-a-cuentas>. [Accedido: 03-mar-2026].
- [15] A. Mascellino, "Telegram Bot Abuse For Phishing Increased By 800% in 2022", InfoSecurity Magazine, 12 ene. 2023. [En línea]. Disponible en: <https://www.infosecurity-magazine.com/news/telegram-bots-phishing-grow-800-in/>. [Accedido: 03-mar-2026].

- [16] F. Heiding, B. Schneier, A. Vishwanath, J. Bernstein, y P. S. Park, "Devising and Detecting Phishing Emails Using Large Language Models", IEEE Access, vol. 12, pp. 42131-42146, 2024, doi: 10.1109/ACCESS.2024.3375882.
- [17] T. Koide, H. Nakano, y D. Chiba, "ChatPhishDetector: Detecting Phishing Sites Using Large Language Models", IEEE Access, vol. 12, pp. 154381-154400, oct. 2024, doi: 10.1109/ACCESS.2024.3483905.
- [18] S. Mahendru y T. Pandit, "SecureNet: A Comparative Study of DeBERTa and Large Language Models for Phishing Detection", en 2024 IEEE 7th International Conference on Big Data and Artificial Intelligence (BD AI), 2024, pp. 160-169, doi: 10.1109/BD AI62927.2024.10692765.
- [19] Kaspersky, "The Telegram phishing market", Securelist, Kaspersky Lab, abr. 2023. [En línea]. Disponible en: <https://securelist.com/telegram-phishing-services/109383/>. [Accedido: 31-mar-2026].
- [20] INKY, "Fresh Phish: Harvesting Your Credentials Using Controversial Telegram Bots", INKY Blog, INKY Technology Corporation, nov. 2024. [En línea]. Disponible en: <https://www.inky.com/en/blog/fresh-phish-harvesting-your-credentials-using-controversial-telegram-bots>. [Accedido: 31-mar-2026].
- [21] python-telegram-bot, "Welcome to python-telegram-bot — v22.6 documentation", python-telegram-bot.org, 2026. [En línea]. Disponible en: <https://docs.python-telegram-bot.org/en/stable/>. [Accedido: 03-mar-2026].
- [22] discord.py, "Welcome to discord.py", discordpy.readthedocs.io, 2026. [En línea]. Disponible en: <https://discordpy.readthedocs.io/en/stable/api.html>. [Accedido: 03-mar-2026].
- [23] Discord.py Documentation, "Gateway Intents", Discord.py, 2024. [En línea]. Disponible en: <https://discordpy.readthedocs.io/en/latest/intents.html>. [Accedido: 03-mar-2026].
- [24] OpenAI, "Best practices for prompt engineering with OpenAI API", OpenAI Platform Documentation, 2023. [En línea]. Disponible en: <https://platform.openai.com/docs/guides/prompt-engineering>. [Accedido: 03-mar-2026].

[25] Google, “Prompt design strategies”, Google AI for Developers (Gemini Documentation), 2023. [En línea]. Disponible en: https://ai.google.dev/docs/prompt_best_practices. [Accedido: 03-mar-2026].

[26] Y. Chen, H. Li, Y. Sui, Y. Liu, Y. He, Y. Song and B. Hooi, “Robustness via Referencing: Defending against Prompt Injection Attacks by Referencing the Executed Instruction”, *CoRR*, vol. abs/2504.20472, Apr. 2025, doi: 10.48550/arXiv.2504.20472.

ANEXO I

Repositorio GitHub con el código Python:

https://github.com/100472127/tfm_llm_bot_phishing_detection.git

ANEXO II



DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA (IAG) EN EL TRABAJO DE FIN DE MÁSTER (TFM)

He usado IA Generativa en mi TFM

Marca lo que corresponda:

SI	NO
----	----

Si has marcado SI, completa las siguientes 3 partes de este documento:

Parte 1: declaración sobre comportamiento legal, ético y responsable

Ten presente que el uso de IAG conlleva unos riesgos y puede generar una serie de consecuencias académicas graves: la evaluación de tu TFM podría verse comprometida si el uso de la IAG comporta la utilización de datos de carácter confidencial, materiales protegidos por derechos de autoría, o datos de carácter personal, y se hace sin cumplir las condiciones exigidas en cada caso (autorización de los interesados, autorización de los titulares, seguimiento de las instrucciones de la Universidad).

Pregunta	
1. En mi interacción con herramientas de IAG he facilitado datos de carácter confidencial contando siempre con la debida autorización de los interesados. La confidencialidad abarca toda información que una persona u organización desea proteger por razones legales, comerciales, de privacidad o estratégicas (como patentes o secretos comerciales).	
SÍ, he usado estos datos con la autorización de los interesados	NO, no he usado datos de carácter confidencial
2. En mi interacción con herramientas de IAG he facilitado materiales protegidos por derechos de autoría contando siempre con la autorización de los respectivos titulares.	

SÍ, he usado estos materiales con autorización de los titulares de derechos de autor; o bien sin ella porque se ajustan a una de las excepciones o límites que permite la ley: • obra en dominio público • obra licenciada (licencias Creative Commons) uso de fragmentos con fines de investigación (derecho de cita)	NO, no he usado materiales protegidos por derechos de autoría
3. En mi interacción con herramientas de IAG he facilitado datos de carácter personal con la debida autorización de los interesados.	
SÍ, he usado estos datos con autorización de los interesados y conforme a las instrucciones contenidas en la guía aprobada por la Universidad	NO, no he usado datos de carácter personal
4. Mi utilización de la herramienta de IAG ha respetado sus términos de uso, así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado, es decir, que produzca una impresión o conocimiento contrario a la realidad de los resultados obtenidos, que suplante mi propio trabajo o que pueda resultar en un perjuicio para las personas.	
SI	NO

Parte 2: declaración de uso técnico

Documentación y redacción:

- Soporte a la reflexión en relación con el desarrollo del trabajo: proceso iterativo de análisis de alternativas y enfoques utilizando la IAG

No se han utilizado herramientas de inteligencia artificial con este enfoque.

- Revisión o reescritura de párrafos redactados previamente

Se han utilizado herramientas de inteligencia artificial de generación de texto, como ChatGPT, y Gemini para corregir la gramática y expresiones de carácter poco profesional con el fin de mejorar la redacción del documento.

- Búsqueda de información o respuesta a preguntas concretas

Se han utilizado herramientas como ChatGPT para buscar información acerca de estudios recientes de phishing y otros ataques de ingeniería social, durante el análisis inicial.

- Búsqueda de bibliografía
- Resumen de bibliografía consultada
- Traducción de texto consultados

Se han utilizado herramientas de inteligencia artificial para generar las referencias bibliográficas siguiendo el estándar IEEE de los documentos y páginas web consultados.

Desarrollar contenido específico

Se ha hecho uso de IAG Generativa como herramienta de soporte para el desarrollo del contenido específico del TFM, incluyendo:

- Asistencia en el desarrollo de líneas de código (programación)

Se ha hecho uso de herramientas de inteligencia artificial como ChatGPT y Gemini para pedir explicaciones y generar código Python.

- Generación de esquemas, imágenes, audios o vídeos

No se han utilizado herramientas de inteligencia artificial con este enfoque.

- Procesos de optimización

Se han utilizado herramientas de inteligencia artificial como ChatGPT y Gemini para arreglar errores y fallos en el código Python.

- Tratamiento de datos: recogida, análisis, cruce de datos...

No se han utilizado herramientas de inteligencia artificial con este propósito.

- Inspiración de ideas en el proceso creativo

No se han utilizado herramientas de inteligencia artificial con este propósito.

- Otros usos vinculados a la generación de puntos concretos del desarrollo específico del trabajo

No se han hecho más usos de herramientas de inteligencia artificial que los mencionados anteriormente.

Parte 3: reflexión sobre utilidad

Por favor, aporta una valoración personal (formato libre) sobre las fortalezas y debilidades que has identificado en el uso de herramientas de IA Generativa en el desarrollo de tu trabajo. Menciona si te ha servido en el proceso de aprendizaje, o en el desarrollo o en la extracción de conclusiones de tu trabajo.

El uso de herramientas de inteligencia artificial representa una ayuda muy valiosa para cualquier persona que realice un trabajo de este tipo, tanto desde el punto de vista técnico como en aspectos relacionados con el diseño y la redacción.

El empleo de herramientas como ChatGPT o Gemini me ha permitido avanzar con mayor rapidez en distintas fases del proyecto que, hace algunos años, me habrían requerido mucho más tiempo. Gracias a la IA resulta más sencillo y ágil consultar dudas relacionadas con el código, además de recibir apoyo para mejorar la redacción y conseguir un estilo más claro y profesional.

No obstante, el uso de estas herramientas en trabajos académicos también puede plantear problemas si se emplean de manera inadecuada. Por ejemplo, los sistemas de IA pueden obtener información de internet sin citar correctamente las fuentes, lo que puede derivar en problemas relacionados con derechos de autor. Asimismo, en el ámbito de la programación, aunque son capaces de generar código con gran facilidad, sigue siendo imprescindible que una persona lo comprenda y lo revise. De este modo, se garantiza que el código funcione correctamente y se evita la aparición de situaciones no previstas por el desarrollador que puedan causar errores o incluso vulnerabilidades de seguridad.